Wise Fine-tuning Pipeline — PRD

문서버전: v0.1 **작성일**: 2026-05-20 **작성자**: Wiseman (bclim@edentns.com) **상태**: Draft — 내부 합의용

> 본 문서는 사용자가 "분야(업종/업태) 선택"만으로 자동으로 LLM 파인튜닝을 진행하고, 학습/평가가 완료된 모델을 다운로드할 수 있게 하는 **자동화 파인튜닝 플랫폼**의 제품 요구 사항(PRD)을 정의한다. 기반 자료는 `.doc/Perplexity-파인튜닝 진행.md`(환경/모델/하드웨어), `.doc/Perplexity-ISIC,KSIC,NACE,NAICS 분류 조사 및 정리.md`(산업 분류 체계)이다.

---

1. 제품 개요

1.1 비전 (Vision)

> "사용자는 파인튜닝의 목적(분야)만 선택한다. 데이터 수집·정제·학습·평가·패키징은 시스템이 자동으로 수행한다."

비기술 사용자도 도메인을 선택하는 것만으로 자신만의 도메인 특화 LLM을 보유할 수 있도록, **데이터 수집 → 정제 → 파인튜닝 → 평가 → 배포(다운로드)** 전 과정을 1-click 자동화한다.

1.2 핵심 가치 제안

항목	가치
도메인 우선 (Domain-first)	KSIC/ISIC/NACE/NAICS 기반 표준 분류 체계로 "분야"를 1차 선택지로 노출
하드웨어 적응 (HW-adaptive)	사용자 시스템 스펙(CPU/GPU/RAM/디스크/OS)을 자동 분석, 최적 파이프라인(전략·모델·양자화·배치 등) 자동 결정
모델 자동 큐레이션	분야·언어·라이선스·HW 적합도 기반 Hugging Face 모델 추천 + 자동 다운로드
데이터 자동 수집/정제	사용자 데이터셋 업로드 또는 인터넷 검색 기반 데이터 수집 + 정제 + 합성 데이터 보강
사용자별 모델 관리	회원별 모델 자산, 학습 이력, 평가 메트릭, 다운로드 링크 제공
이중 배포	데스크탑앱(로컬 GPU 친화) + 웹앱(서버/클라우드 친화) 동시 제공

1.3 비목표 (Non-goals)

- Pretrain (사전학습)은 제공하지 않는다. Fine-tuning(SFT/LoRA/QLoRA) 및 DPO/ORPO 수준까지만.
- 멀티모달(이미지/음성) v1 범위 외. (텍스트 LLM 우선, v2 확장 고려.)
- AGI급 일반 챗봇 경쟁이 아닌, **도메인 특화 모델 생성기**이다.

---

2. 사용자 (Personas)

2.1 일반 사용자 (End User)

- **Beginner**: 비기술 사용자. UI 따라가며 분야·데이터·모델만 선택.
- **Pro**: 데이터 사이언티스트. 하이퍼파라미터·전략 직접 오버라이드 가능.

2.2 관리자 (Admin)

- **Ops Admin**: 잡 큐, GPU 점유, 디스크/네트워크 사용량 모니터링. 자원 회수.

- **Service Admin**: 사용자 관리, 모델 카탈로그 관리, 분야 카탈로그 큐레이션.

---

3. 도메인 카테고리 — KSIC/ISIC/NACE/NAICS 기반

3.1 분류 체계 전략

분야(Domain) 선택 UI는 KSIC 대분류(A~U, 21개)를 1단계, 중분류(2자리)를 2단계로 노출한다. KSIC는 ISIC를 기반으로 하므로 국제 표준 ISIC/NACE/NAICS와 동시 매핑한다.

레벨	UI 노출	코드 체계
L1 (대분류)	“분야” 1차 선택 (A~U)	KSIC Section / ISIC Section / NACE Section / NAICS 2-digit
L2 (중분류)	“업종” 2차 선택 (2자리)	KSIC Division / ISIC Division / NACE Division / NAICS Subsector
L3 (세부)	“종목” 자유 텍스트 / 선택	KSIC 소·세·세세분류

3.2 카테고리별 파이프라인 프리셋

각 L1/L2 카테고리에 대해 **추천 베이스 모델 / 추천 데이터 소스 / 추천 학습 전략**을 프리셋으로 정의한다.

L1 카테고리 (KSIC)	추천 베이스 모델 (예시)	추천 데이터 소스	권장 전략
C 제조업	Qwen2-7B-Instruct, Mistral-7B-Instruct	사내 매뉴얼, 품질/안전 규격, KOSHA 공개자료	QLoRA + 도메인 어댑터
G 도매·소매	Llama-3-8B-Instruct, Qwen2-7B	상품 카탈로그, 고객 리뷰, FAQ	QLoRA + Instruction 튜닝
I 숙박·음식점	Gemma-2-9B-it, Phi-3-medium	메뉴/리뷰/관광공사 공개 데이터	QLoRA + 톤·매너 튜닝
J 정보통신	CodeLlama-13B, DeepSeek-Coder-6.7B, Qwen2.5-Coder	사내 코드, 기술 문서, GitHub 공개	QLoRA + Code-specific
K 금융·보험	Llama-3-8B-Instruct, Qwen2-14B	공시자료, 약관, 금감원 공개	LoRA + 안전성 평가 강화
M 전문·과학·기술	Llama-3-8B-Instruct, Mistral-7B	논문(arXiv), 특허, 표준 문서	LoRA + 긴 컨텍스트
P 교육	Gemma-2-9B-it, Llama-3-8B	교과서, 문제은행, EBS 공개	QLoRA + 난이도 조절
Q 보건·사회복지	Llama-3-8B-Instruct, BioMistral, Meditron	의학논문, 임상가이드라인, AI Hub 의료	QLoRA + 안전성/근거성 평가 강화
F 건설	Mistral-7B-Instruct, Qwen2-7B	건설표준시방서, KS 표준	QLoRA
기타 (A,B,D,E,H,L,N,O,R,S,T,U)	기본 프리셋 (Llama-3-8B-Instruct + QLoRA)	일반 공개 데이터 + 사용자 업로드	QLoRA

> 카테고리 → 프리셋 매핑은 DB(`category_preset`) 테이블에 저장하고 관리자가 변경 가능하다.

3.3 카테고리 데이터 모델 (요약)

```
industry_category
  - id (uuid)
  - ksic_section (A~U)
  - ksic_division (01~99)
```

```
  - isic_division
  - nace_division
  - naics_code
  - ko_name, en_name
  - parent_id (self-ref)

category_preset
  - category_id (FK)
  - base_model_recommendations (jsonb: HF model ids + priority)
  - dataset_sources (jsonb: urls, search keywords, ai-hub ids)
  - default_strategy (enum: full_ft | lora | qlora | dpo)
  - default_hparams (jsonb)
```

---

4. 파인튜닝 파이프라인 (8 Stage)

`.doc/Perplexity-파인튜닝 진행.md` 의 8단계를 자동화 워크플로우로 변환한다.

```
[1.Init/Goal] → [2.HW Probe] → [3.Env Setup] → [4.Model Pick & Download]
   → [5.Data Collect/Clean] → [6.Train Config] → [7.Train Run]
   → [8.Eval/Iterate] → [9.Package & Deliver]
```

4.1 Stage 1 — 목표 정의 (Init)

- 입력: 사용자가 선택한 카테고리(L1/L2), 태스크 타입(chat/qa/summary/classification/code), 모델 용도 설명(자유 텍스트).
- 출력: `job_spec.yaml` (모든 후속 단계의 입력 명세).

4.2 Stage 2 — 하드웨어 프로빙 (HW Probe)

- 시스템 사양 자동 수집 후 "가능한 모델 크기 × 전략"을 계산.
- 상세는 §5 참고.

4.3 Stage 3 — 환경 설정 (Env Setup)

- **Python 환경**: `uv venv` 기반 가상환경 자동 생성 (Python 3.11 권장).
- **드라이버 검증**: NVIDIA CUDA / AMD ROCm / Apple Metal(MPS) / CPU-only 자동 판별.
- **필수 라이브러리 설치** (자동): `torch` (CUDA/MPS/CPU 빌드 자동 선택), `transformers>=4.45`, `datasets`, `accelerate`, `peft`, `bitsandbytes`, `trl`, `unsloth` (옵션), `evaluate`, `tensorboardX`.
- **OS별 분기**:
  - Linux+CUDA: 기본 경로.
  - Windows: WSL2 + Ubuntu 권장 자동 가이드, 미설치 시 안내.
  - macOS (Apple Silicon): MPS 백엔드, `bitsandbytes` 미지원 → `mlx-lm` 또는 `unsloth(MPS)` 대체.

4.4 Stage 4 — 베이스 모델 선택 및 다운로드

- 후보 모델 자동 추천: (카테고리, 언어, HW 적합도, 라이선스) 기준 정렬.
- 사용자가 후보 중 하나 선택 (또는 자동 모드면 1순위 자동 채택).
- Hugging Face Hub 자동 다운로드 (`huggingface_hub` 사용, 토큰 필요 시 안전 저장소에서 로드).
- 로컬 모델 캐시 디렉토리: `~/.wfp/models/<org>/<model>/<revision>`.

4.5 Stage 5 — 데이터 수집·정제

사용자가 데이터 소스를 다음 중 선택한다.

모드	동작
A. 보유 데이터 업로드	사용자가 JSONL/CSV/Parquet 업로드. 자동 스키마 추론 + Instruction 포맷 변환.
B. 디렉토리 선택	데스크탑앱에서 로컬 디렉토리 지정. PDF/DOCX/HTML/MD/TXT 자동 파싱.
C. 인터넷 검색 수집	키워드 + 카테고리 기반 자동 검색 → 화이트리스트 도메인 크롤링 + 정제.
D. 하이브리드	A/B + C 결합. + 합성 데이터(self-instruct) 옵션.

정제 파이프라인 (공통): 1. Deduplication (MinHash / SimHash) 2. Language Filter (FastText langid) 3. Quality Filter (perplexity 기반 + 길이/공백 비율 휴리스틱) 4. PII/민감정보 제거 (정규식 + Presidio) 5. Instruction 포맷 변환 (`{"instruction":..., "input":..., "output":...}` 또는 chat-messages) 6. Train/Val/Test 분할 (기본 8:1:1)

합성 데이터(옵션): Teacher 모델(API 또는 강한 로컬 모델)로 instruction-response 쌍 생성 (Self-Instruct, Evol-Instruct).

4.6 Stage 6 — 학습 설정 (Train Config)

HW 프로빙 결과를 토대로 자동 결정: - 전략 (Full FT / LoRA / QLoRA / DPO) - 양자화 (4bit/8bit/none) - max_seq_length, batch_size, gradient_accumulation - learning_rate, num_epochs, warmup_ratio - LoRA r, alpha, target_modules

사용자는 "추천값 사용" 또는 "고급 사용자 → 직접 수정" 선택.

4.7 Stage 7 — 학습 실행 (Train Run)

- 실행 엔진: `transformers.Trainer` + `trl.SFTTrainer` 기본. 옵션으로 `unsloth` (속도 2x).
- 분산학습: 단일 노드 다중 GPU 시 `accelerate launch` 자동 구성.
- 체크포인트: N step 마다 저장, best-by-eval-loss 자동 유지.
- 모니터링: Tensorboard + 자체 메트릭 스트림(웹소켓으로 프론트 전송).

4.8 Stage 8 — 평가 (Eval)

평가 타입	도구/메트릭
자동 메트릭	perplexity, BLEU, ROUGE, exact-match, F1 (태스크별)
벤치마크	KMMLU, KoBEST, MMLU, HellaSwag, ARC (카테고리별 일부 선택)
도메인 평가셋	카테고리 프리셋이 보유한 held-out 평가셋
LLM-as-Judge	GPT-4o/Claude-3.5 또는 강한 로컬 모델로 정성 평가
안전성/편향	bias, toxicity (Detoxify, HolisticBias)

기준 미달 시 자동 재시도 (epoch 조정 또는 데이터 보강) — 최대 N회.

4.9 Stage 9 — 패키징·배포 (Package & Deliver)

- 아티팩트:
  - LoRA 어댑터 (`adapter_model.safetensors`)
  - 머지 weights (옵션)
  - GGUF 변환본 (옵션, `llama.cpp` 사용)
  - `model_card.md` 자동 생성 (카테고리, 데이터 출처, 평가 결과, 라이선스)
- 다운로드: 사용자 대시보드에서 zip 또는 stream 다운로드. 데스크탑앱은 로컬 디렉토리 직접 복사.

---

5. 하드웨어 적응 (HW-Adaptive Pipeline)

5.1 HW 프로빙 — 오픈소스 선정

영역	후보 라이브러리	선택	근거
시스템 정보 (OS/CPU/RAM/Disk)	`psutil` / `platform` / `py-cpuinfo`	**psutil + py-cpuinfo + platform**	크로스플랫폼, 안정적, 의존성 적음
NVIDIA GPU	`pynvml` / `gpustat` / `nvidia-ml-py`	**pynvml**	NVIDIA 공식 ML 바인딩, VRAM/온도/사용률 정확
AMD GPU	`pyrsmi` / `rocm-smi`	**pyrsmi** (호출 가능 시)	ROCm 환경에서만 사용
Apple Silicon	`subprocess` + `system_profiler` + `mlx` 감지	**macOS native**	Metal/Unified Memory 검사
토치-수준 검증	`torch.cuda.is_available()`, `torch.backends.mps.is_available()`	**torch**	최종 백엔드 확정
디스크 I/O	`psutil.disk_io_counters`	**psutil**	학습 중 병목 감지
네트워크	`psutil.net_if_stats`	**psutil**	HF 다운로드 ETA 추정

> 보조 도구로 `nvitop`(터미널 UI, 디버깅용), `gpustat`(짧은 요약), `lshw`(Linux 한정 상세) 를 채택하되, 프로그래밍 인터페이스는 위 표를 정본으로 한다.

5.2 HW 점수 → 파이프라인 매핑

프로빙 결과로 다음 능력 점수를 산출한다.

```
caps = {
  os: "linux" | "windows" | "darwin",
  gpu: { vendor, vram_gb, count, compute_capability },
  cpu: { cores, threads, avx2, avx512 },
  ram_gb,
  disk_free_gb,
  network_mbps
}
```

`caps` → `recommended_plan` 결정 규칙 (요약):

조건	권장 모델 크기	전략	양자화
VRAM ≥ 80GB (A100/H100)	70B까지	LoRA / Full FT(소형)	bf16 또는 4bit
VRAM 48GB	34B까지	LoRA / QLoRA	4bit
VRAM 24GB (3090/4090)	13B까지	QLoRA	4bit
VRAM 16GB (4070Ti/4080)	8B (7~13B QLoRA 가능)	QLoRA	4bit
VRAM 12GB (3060/4060)	7~8B	QLoRA	4bit
VRAM 6~8GB	1~3B (또는 7B 매우 보수적)	QLoRA + 짧은 seq	4bit
Apple Silicon (Unified 32GB+)	7B	unsloth(MPS) / mlx-lm	4bit
Apple Silicon (16GB)	1~3B	mlx-lm	4bit
GPU 없음, RAM 32GB+	1B 이하 (학습 비추, 추론만 권장)	CPU LoRA / 클라우드 안내	4bit

규칙은 `ops/rules/hw_to_plan.yaml`에 외부화하여 관리자가 조정 가능하게 한다.

5.3 상세 비교 — HW 프로빙 라이브러리

라이브러리	플랫폼	GPU	의존성	라이선스	채택
psutil	전 플랫폼	X	무	BSD	☒ 기본
py-cpuinfo	전 플랫폼	X	무	MIT	☒ CPU 명령어 detect
pynvml	NVIDIA만	☒	NVIDIA driver	BSD	☒ NVIDIA
GPUtil	NVIDIA만	☒	nvidia-smi	MIT	☒ (pynvml로 대체)
gpustat	NVIDIA	☒	pynvml	MIT	△ (CLI 보조)
nvitop	NVIDIA	☒	pynvml	Apache-2.0	△ (디버깅)
pyrsmi	AMD ROCm	☒	rocm-smi	MIT	☒ AMD 한정
lshw / dmidecode	Linux	△	OS 패키지	GPL	△ (서버 한정 보조)
torch.cuda / .mps	전 플랫폼	☒	PyTorch	BSD	☒ 최종 검증

---

6. 사용자 플로우 (Wizard)

6.1 신규 작업 마법사 (5 Steps)

```
[Step 1] 분야 선택
  - L1 (KSIC Section A~U) 리스트 카드
  - L2 (Division) 그리드
  - "비슷한 작업 검색" 자유 텍스트
        ↓
[Step 2] 모델 선택
  - 카테고리·HW 기반 추천 Top 5 카드 (모델명, 크기, 라이선스, 한국어 적합도, HW 적합도)
  - "자동 선택" 또는 "수동 선택"
        ↓
[Step 3] 데이터 선택
  - (A) 업로드 (B) 디렉토리 (C) 인터넷 검색 (D) 합성 생성
  - 미리보기 100건 + 자동 정제 결과 통계 표시
        ↓
[Step 4] 학습 설정
  - 권장값 카드 + "고급" 토글
  - 예상 학습 시간 / 예상 디스크 / 예상 메모리 표시
        ↓
[Step 5] 실행 확인
  - 요약 리뷰 → "시작"
        ↓
[진행 모니터링 페이지]
  - Stage별 진행률, 로그 스트림, 손실 그래프
  - 완료 시 알림 + 다운로드 버튼
```

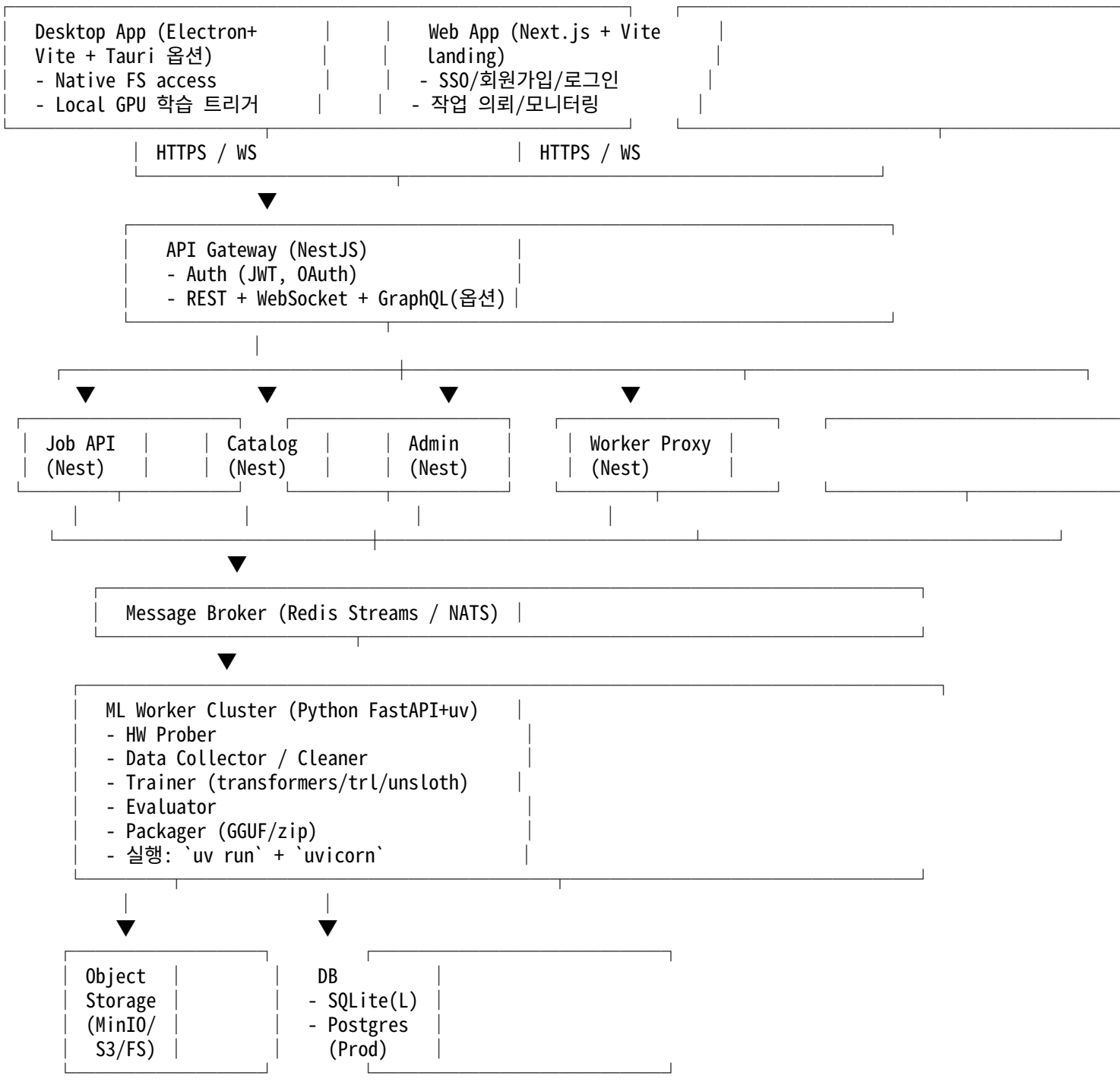
6.2 자동 모드 (1-Click)

Step 1만 사용자가 선택 → Step 2~5 모두 자동 결정. “고급 사용자” 토글 없으면 기본값.

---

7. 시스템 아키텍처

7.1 전체 컴포넌트 다이어그램



7.2 컴포넌트별 책임

컴포넌트	기술	책임
frontend/desktop	Electron + Vite + React (또는 Tauri)	로컬 파일 접근, 로컬 GPU 직접 실행 모드, 진행 UI

컴포넌트	기술	책임
frontend/web	Next.js (App Router) + Vite-powered libs	회원가입/로그인/대시보드/관리자 UI
backend/api-gateway	NestJS (Node)	인증, REST/WS, 권한, 라우팅
backend/job-service	NestJS	작업 생성·조회·이력
backend/catalog-service	NestJS	카테고리, 모델 카탈로그, 프리셋
backend/admin-service	NestJS	사용자/리소스/모니터링/회수
backend/worker-proxy	NestJS	워커 큐 디스패치, 워커 헬스 체크
ml-worker	Python 3.11 + FastAPI + uv 실행 + uvicorn 서버	실제 ML 작업 수행
database/local	SQLite (PRAGMA journal_mode=WAL)	데스크탑앱·개발 환경
database/prod	PostgreSQL 16	운영 환경
broker	Redis Streams (기본) / NATS (대안)	잡 큐, 이벤트 스트림
storage	MinIO (개발) / S3 (운영) / 로컬 FS (데스크탑)	모델/데이터셋/체크포인트
ops	Docker Compose, Makefile, shell scripts, GitHub Actions	빌드/배포/CI

7.3 모놀리포 디렉토리 구조

```
Wise-Finetuning-Pipeline/
├── frontend/
│   ├── web/                # Next.js 앱
│   │   ├── app/
│   │   ├── components/
│   │   └── package.json
│   ├── desktop/            # Electron(or Tauri) 앱
│   │   ├── src/
│   │   ├── electron/
│   │   └── package.json
│   └── shared/             # 공용 UI/타입 (Vite 라이브러리 빌드)
├── backend/
│   ├── api-gateway/        # NestJS
│   ├── job-service/
│   ├── catalog-service/
│   ├── admin-service/
│   └── worker-proxy/
├── ml-worker/             # Python (uv 관리)
│   ├── pyproject.toml      # uv + ruff + pytest
│   ├── src/wfp_worker/
│   │   ├── hw_probe/
│   │   ├── data/
│   │   ├── train/
│   │   ├── eval/
│   │   ├── pack/
│   │   └── server.py       # FastAPI + uvicorn
│   └── tests/
├── database/
│   ├── migrations/         # Prisma or TypeORM (Node) + Alembic (Python 측 보조)
│   ├── seed/
│   └── schemas/
├── ops/
│   ├── docker/
│   │   ├── compose.dev.yaml
```

```
│   │   ├── compose.prod.yaml
│   │   ├── api.Dockerfile
│   │   ├── worker.Dockerfile
│   │   └── web.Dockerfile
│   ├── scripts/            # bash: bootstrap, release, gpu-check, ...
│   ├── rules/              # hw_to_plan.yaml 등
│   └── ci/                 # GitHub Actions workflow 파일
├── docs/
│   └── ...
├── .doc/                  # PRD, 조사자료
├── Makefile
└── README.md
```

7.4 데이터 흐름 (Job Lifecycle)

```
User → Web/Desktop → API Gateway → Job Service
   ↓
Job 생성 (state=PENDING) → DB 기록 + Broker로 push
   ↓
ML Worker pull → HW Probe → Stage 진행
   ↓ (Stage 전이마다)
Broker로 이벤트 publish → API Gateway WS → Frontend 실시간 표시
   ↓ (Stage 9 완료)
Storage 업로드 → DB(state=DONE) → User에게 알림 + 다운로드 URL
```

7.5 웹 UI 오픈소스 비교 및 선정

본 절은 ml-worker 파이프라인(`ml-worker/`)을 비기술 사용자가 사용할 수 있도록 하는 **웹 UI 레이어**의 오픈소스 후보를 비교·평가한다.

7.5.1 요구사항 (UI 측면)

ID	요구사항	근거
UI-1	KSIC 대분류(A~U) 카드 UI 1차 선택 + 중분류 그리드	§ 6.1 Step 1
UI-2	5-step 위자드 + 상세 진행 모니터링 페이지	§ 6.1
UI-3	멀티 사용자, RBAC(USER/ADMIN), 자원 단위 ACL	§ 9, § 10
UI-4	실시간 진행 스트림(WebSocket) + 손실 그래프	§ 4.7, § 6.1
UI-5	어드민 대시보드 (GPU 점유율, 잡 큐, 회수 트리거)	§ 10.1~10.5
UI-6	다국어 (한/영) — v1 한국어 우선	§ 12 i18n
UI-7	데스크탑(Electron/Tauri)에서 동일 UI 재사용	§ 7.2
UI-8	WCAG 2.1 AA 접근성	§ 12

7.5.2 후보 오픈소스 비교

후보	라이선스	특화 영역	위자드 UI	멀티유저/RBAC	KSIC 카드 커스텀	학습 진행 그래프	데스크탑 임베드	한국어	통합 난이도
Next.js 14 (App Router) + shadcn/ui + TanStack Query + Tail-wind	MIT	범용 SaaS	◎	◎	◎	◎ (Recharts/Tremor)	◎	◎	High (코딩)
Gradio (github.com/gradio-app/gradio)	Apache-2.0	ML 데모/추론 인터페이스	△ Block 모드만	△ Hugging Face Spaces 의존	△ HTML/CSS 제한	○ (gr.Plot)	△ iframe만	○	Low
Streamlit (github.com/streamlit/streamlit)	Apache-2.0	데이터 사이언스 앱	△ 페이지 분리	△ 외부 인증 필요	△ 제한적	◎	△	○	Low
LLaMA-Factory Web-bUI (github.com/hiyouga/LLaMA-Factory)	Apache-2.0	LLM 파인튜닝 GUI	○	☒ 단일 세션	☒ 코드 포크 필요	○	☒	△	Med (포크)
H2O LLM Stu-dio (github.com/h2oai/h2o-llmstudio)	Apache-2.0	LLM 파인튜닝 GUI	○	△ 단일 인스턴스	☒	○	☒	△	Med
Open We-bUI (github.com/open-webui/open-webui)	MIT	Chat / RAG	☒	◎	☒ 챗UI 위주	☒	△	○	Low
Refact.ai (github.com/smallcloudai/refact)	BSD-3	Code LLM 학습	△	△	☒	○	☒	△	Med
AnythingLLM (github.com/Mintplex-Labs/anything-llm)	MIT	챗봇/RAG	☒	◎	☒	☒	○	○	Low

평가 기준 (가중치): - 위자드 UI / KSIC 커스텀 / 멀티유저: 본 PRD의 1차 가치 → **가중치 高** - 학습 진행 그래프 / 어드민 대시보드: 운영 필수 → **가중치 中** - 한국어 / 통합 난이도: 부가 → **가중치 低**

7.5.3 최종 추천 — Hybrid (Next.js + Gradio)

주 UI: Next.js 14 + shadcn/ui + TanStack Query + Tailwind
보조 UI: Gradio (학습 완료 모델 추론 시연용 iframe)

역할	도구	설명
메인 사용자 UI	Next.js (App Router)	위자드, 대시보드, 어드민, 회원/RBAC
컴포넌트 라이브러리	**shadcn/ui**	Radix 기반, 디자인 토큰 커스텀 가능, MIT, 코드 소유
폼/검증	**react-hook-form + zod**	KSIC 카테고리·태스크 타입 강제 검증
데이터 페칭	**TanStack Query**	잡 상태 폴링 + WebSocket 캐시 동기화
차트	**Tremor + Recharts**	잡 큐, GPU 점유율, 손실 그래프
WebSocket	**native + zustand store**	`/ws/{job_id}` 실시간 진행 스트림
i18n	**next-intl**	한/영 메시지 카탈로그
인증	**Auth.js (NextAuth)**	JWT + OAuth (Google/GitHub/Kakao)
데스크탑 임베드	**Tauri** WebView	동일 Next.js 빌드 재사용 (Electron 대비 경량)
추론 시연	**Gradio (Python)**	Stage 9 완료 모델을 iframe으로 즉시 데모
디자인 시스템	**Tailwind CSS v3.4 + Radix Colors**	다크모드·접근성 기본

추천 근거: 1. **UI-1 KSIC 카드 + UI-2 위자드**: 비기술 사용자 1차 진입점. shadcn/ui Card/Stepper 컴포넌트로 자유 구성 가능. Gradio·Streamlit은 데모 도구 성격이라 카드 그리드 표현이 제한적. 2. **UI-3 멀티유저 RBAC**: Next.js + Auth.js + NestJS gateway 조합이 표준. Gradio Spaces 인증은 본 도메인 사용 사례에 부적합. 3. **UI-4 WS 진행 그래프**: TanStack Query + Recharts 조합이 NestJS Gateway WS와 자연스럽게 결합. Streamlit도 가능하나 멀티유저 동시 접속 시 세션 격리 문제. 4. **UI-7 데스크탑 재사용**: Tauri로 동일 Next.js 빌드를 WebView에 임베드 → 코드 1벌, 배포 2채널. 5. **Gradio 보완 가치**: 파인튜닝 직후 사용자가 즉시 모델을 테스트하고 싶을 때, Gradio `ChatInterface`로 30줄에 추론 페이지 생성 가능. iframe으로 Next.js 페이지에 끼워 넣음. 자체 구현 비용 대비 효과 명확.

기각 사유: - **LLaMA-Factory / H2O LLM Studio**: 파인튜닝에 특화되어 있으나 단일 사용자·단일 세션 가정. 멀티 테넌트 + KSIC 카테고리 + 어드민 회수 정책을 위해 포크·재작성 비용 > 자체 구축. - **Open WebUI / AnythingLLM**: 챗봇·RAG 중심. 학습 파이프라인 UI 부재. - **Refact.ai**: 코드 LLM 전용 + 자체 IDE 통합 지향. 본 프로젝트의 카테고리 범위(KSIC 21개)와 맞지 않음.

7.5.4 페이지 구조 (Next.js App Router)

```
frontend/web/app/
├── (marketing)/              # 랜딩, 가격, 약관 (공개)
│   ├── page.tsx
│   └── pricing/page.tsx
├── (auth)/
│   ├── login/page.tsx
│   ├── signup/page.tsx
│   └── callback/page.tsx     # OAuth 콜백
├── (app)/                    # 인증 필요 영역
│   ├── dashboard/page.tsx    # 잡 리스트 + 카드 (사용자별)
│   ├── jobs/
│   │   ├── new/              # 5-step 위자드
│   │   │   ├── step-1-category/page.tsx
│   │   │   ├── step-2-model/page.tsx
│   │   │   ├── step-3-data/page.tsx
│   │   │   ├── step-4-config/page.tsx
│   │   │   └── step-5-confirm/page.tsx
│   │   └── [id]/
│   │       ├── page.tsx      # 진행 모니터링 (WS 스트림)
│   │       ├── playground/   # Gradio iframe
│   │       └── artifacts/    # 다운로드
```

```
│   └── settings/page.tsx
├── (admin)/                  # ADMIN 권한 필요
│   ├── users/page.tsx
│   ├── jobs/page.tsx
│   ├── resources/page.tsx    # GPU/디스크 점유율
│   ├── catalog/              # 카테고리·프리셋 CRUD
│   └── audit/page.tsx
└── api/                      # BFF 라우트 (NestJS gateway 호출)
```

7.5.5 Gradio 보조 UI 사양

항목	사양
위치	`ml-worker/src/wfp_worker/playground/app.py`
트리거	Stage 9 완료 후 `playground.enabled = true` 플래그
노출 경로	`/api/playground/{job_id}` (NestJS 프록시) → Gradio `/run/predict`
인증	Gradio auth 또는 NestJS 프록시 레벨에서 JWT 검증
컴포넌트	`gr.ChatInterface` + 시스템 프롬프트 편집 + 토큰 수 표시
격리	잡별 독립 프로세스 (자동 종료 60분 유휴)

---

8. 데이터베이스 설계

8.1 이중 DB 정책

환경	DB	용도
데스크탑 / 로컬 개발	**SQLite** (file: `~/.wfp/wfp.db`)	단일 사용자, 임베디드
운영 / 멀티 사용자	**PostgreSQL 16**	동시성, 권한, FK 강제

ORM 계층(Prisma 또는 TypeORM)에서 동일 스키마 정의 후 두 엔진 동시 지원. JSON 컬럼은 SQLite=`TEXT(JSON)`, Postgres=`JSONB` 매핑.

8.2 핵심 엔티티

```
users
  id, email, password_hash, role(USER|ADMIN), created_at, last_login

industry_category
  id, ksic_section, ksic_division, isic_division, nace_division,
  naics_code, ko_name, en_name, parent_id

category_preset
  id, category_id(FK), base_models(jsonb), data_sources(jsonb),
  strategy, hparams(jsonb), updated_by, updated_at

models
  id, hf_repo_id, revision, family, size_params, language,
  license, downloaded_path, status, downloaded_at

datasets
  id, user_id, source_type(UPLOAD|DIR|WEB|SYNTH), uri,
  num_records, schema(jsonb), created_at
```

```
jobs
  id, user_id, category_id, model_id, dataset_id,
  spec(jsonb), state(PENDING|RUNNING|EVAL|DONE|FAILED|CANCELLED),
  stage, hw_caps(jsonb), plan(jsonb),
  started_at, ended_at, artifacts(jsonb)

job_events
  id, job_id, ts, level, stage, message, metrics(jsonb)

artifacts
  id, job_id, kind(adapter|merged|gguf|zip|model_card), uri,
  size_bytes, sha256, created_at

resources
  id, host, gpu_uuid, vram_gb, allocated_job_id, last_heartbeat

audit_logs
  id, user_id, action, target, before(jsonb), after(jsonb), ts
```

8.3 마이그레이션 전략

- 단일 마이그레이션 소스 (Prisma migrate 권장). Python ml-worker는 read-only로 DB 접근하거나 별도 API 통해 접근.

---

9. 인증/권한 (Auth)

- 회원가입: 이메일+비밀번호 (Argon2id 해시) + 이메일 인증.
- 소셜 로그인 (v1.5+): Google, GitHub, Kakao.
- 인증 방식: JWT (access 15분 + refresh 14일) + httpOnly 쿠키 (웹), Keychain/Credential Vault (데스크탑).
- 권한: RBAC (USER / ADMIN). 추가로 리소스 단위 ACL (사용자는 본인 잡만 조회/취소).

---

10. 관리자 기능

10.1 대시보드

- 전체 사용자/작업/리소스 현황 카드.
- 큐 길이, 평균 대기 시간, 평균 학습 시간 그래프.
- GPU별 점유율, VRAM 사용량 (pynvml 스트림).

10.2 작업 모니터링

- 잡 리스트 필터 (사용자/카테고리/상태/기간).
- 잡 상세: 실시간 로그, 메트릭, 체크포인트 목록.
- 강제 취소 / 우선순위 변경.

10.3 자원 회수 (Reaper)

- 정책:
  - heartbeat 끊김 N분 이상 → 잡 FAILED 처리 + 자원 해제.
  - 일정 기간 미접속 사용자의 아티팩트 → soft-delete (grace period 후 hard-delete).
  - 디스크 사용량 임계치 초과 시 오래된 체크포인트부터 회수.
- 구현: 별도 cron 워커 (`ops/scripts/reaper.py` + Docker Compose의 cron 사이드카) + 관리자 수동 트리거.

10.4 카탈로그 관리

- 카테고리/프리셋 CRUD.
- 모델 카탈로그 (HF repo id, 라이선스, 기본값) CRUD.
- 데이터 소스 화이트리스트 관리.

10.5 감사 로그

- 모든 관리자 액션은 `audit_logs` 적재.

---

11. 기술 스택 및 표준

11.1 언어/런타임 버전

영역	도구	버전
Python	uv	최신
Python runtime	CPython	3.11.x (3.12 가능)
ASGI 서버	uvicorn	최신
Node	Node.js	20.x LTS
패키지 매니저 (Node)	pnpm	9.x
웹 프론트	Next.js (App Router)	14.x+
데스크탑	Electron 또는 Tauri	최신 LTS
백엔드	NestJS	10.x+
프론트 번들	Vite	5.x+
컨테이너	Docker / Compose v2	최신
DB	PostgreSQL / SQLite	16 / 3.45+
Broker	Redis	7.x
Object Storage	MinIO / S3 호환	최신

11.2 코딩 표준 / Notation

- **Python**: PEP8 + `ruff` (lint+format), `mypy --strict` (가능 범위), `pytest`, 모듈/함수는 snake_case, 클래스는 PascalCase.
- **TypeScript**: `eslint + prettier`, `tsc --strict`, 함수/변수 camelCase, 타입/클래스 PascalCase, 상수 UPPER_SNAKE.
- **API**: REST는 kebab-case 경로 (`/api/v1/jobs/{id}`). 응답 키는 camelCase. 에러 응답은 RFC 7807 (problem+json).
- **DB**: 테이블·컬럼 snake_case, PK는 `id`(uuid v7), FK는 `<table>_id`.
- **Git**: Conventional Commits (`feat:`, `fix:`, `chore:` 등). 브랜치 `feat/<slug>`, `fix/<slug>`. PR 템플릿 + 최소 1 reviewer.
- **버전**: SemVer. 릴리스 태그 `v0.1.0`.
- **로깅**: 구조화 로그 (JSON), 필드 표준: `ts`, `level`, `service`, `job_id`, `user_id`, `msg`, `extra`.
- **에러 코드**: `WFP-XXXX` 형태 (예: `WFP-0301` = HF 다운로드 실패).

11.3 빌드/운영 (ops)

- `Makefile` 진입점:
  - `make bootstrap` — 전체 의존성 설치 (uv sync + pnpm install)
  - `make dev` — Docker Compose dev 스택 기동
  - `make worker` — 로컬 ml-worker 단독 실행
  - `make test`, `make lint`, `make build`, `make release`
- `ops/scripts/`:
  - `bootstrap.sh` — OS 검사, uv/pnpm 설치, 드라이버 가이드

  - `gpu-check.sh` — `nvidia-smi/rocm-smi/system_profiler` 검사
  - `release.sh` — 버전 태깅 + 컨테이너 빌드/푸시
  - `reaper.sh` — 자원 회수 trigger
- `compose.dev.yaml`: api, web, worker(GPU), postgres, redis, minio, mailhog.
- `compose.prod.yaml`: 위 + reverse-proxy(traefik), backup 잡.
- CI: GitHub Actions — lint/test/build → 컨테이너 빌드 → 스캔(trivy) → 배포.

---

12. 비기능 요구사항 (NFR)

항목	요구
성능	평균 잡 큐 대기 < 30초 (워커 가용 시). 학습 throughput은 HW 최대치 80%+ 활용.
확장성	워커 horizontal scaling (Compose 또는 K8s). 잡 수 1k/일까지 단일 노드.
가용성	운영 99%+. 워커 실패 시 자동 재시도(1회) 후 사용자 알림.
보안	TLS everywhere, 시크릿은 `.env` + Vault(운영). HF 토큰/DB 비밀번호 코드 노출 금지. PII 정제 의무.
관측성	메트릭(Prometheus), 로그(JSON, Loki), 트레이스(OpenTelemetry).
백업	DB 일 1회 dump → S3. 아티팩트는 사용자 다운로드 후 30일 보관 (정책 가변).
i18n	UI 한/영 지원 (v1: 한국어 우선, v1.5: 영어).
접근성	WCAG 2.1 AA 목표.

---

13. 라이선스 / 컴플라이언스

- HF 모델 사용 시 각 모델의 라이선스(특히 Llama, Qwen, Gemma) 사용자에게 명시 + 동의 체크박스.
- 인터넷 수집 데이터는 robots.txt 준수 + 도메인 화이트리스트만.
- 사용자 업로드 데이터는 사용자 소유, 시스템은 학습 목적으로만 처리.
- PII/민감정보 자동 마스킹 + 사용자 옵트인 처리.

---

14. 로드맵 (Milestones)

마일스톤	범위	목표 시점
M0 — Spec Lock	본 PRD 확정, 아키텍처 합의	작성 후 +1주
M1 — Skeleton	모놀리포 구조, 인증, Job CRUD, 빈 ML 워커 + WS 진행 표시	+4주
M2 — Single-Path FT	KSIC J/M 카테고리 1개 모델(Qwen2-7B QLoRA) end-to-end 자동화	+8주
M3 — HW Adapter	HW 프로빙 + 자동 플랜 결정 (NVIDIA/Apple Silicon 우선)	+10주
M4 — Multi-category	KSIC 상위 카테고리 8종 프리셋 + 데이터 수집 모드 A/B/C/D	+14주
M5 — Admin & Reaper	관리자 대시보드, 자원 회수, 감사 로그	+16주
M6 — Web + Desktop Parity	웹/데스크탑 두 클라이언트 기능 동등	+18주
M7 — Beta	외부 사용자 베타, 안정성/문서화	+22주

마일스톤	범위	목표 시점
v1.0	GA	+26주

---

15. 리스크 & 미해결 이슈

리스크	영향	완화책
HF 모델 라이선스 변동	사용 불가	카탈로그에 라이선스 monitor, 폴백 모델 지정
GPU 가용성 부족	잡 적체	클라우드 burst 옵션 (RunPod/vast.ai 어댑터)
인터넷 수집 데이터 품질/저작권	법적 리스크	화이트리스트 + 사용자 명시 동의 + 자동 정제
Apple Silicon 학습 호환성	macOS UX 저하	unsloth(MPS)/mlx-lm 이중 백엔드, 안 되는 모델은 사전 차단
데이터 PII 누락	보안/법적	Presidio + 정규식 다중 레이어, 학습 전 사용자 미리보기 강제
데스크탑 자동 업데이트	운영 부담	Electron/Tauri auto-update, 서버 호환성 헤더

---

16. 부록

16.1 카테고리 매핑 참고

- KSIC 대분류 A~U (21개) ↔ ISIC Section / NACE Section ↔ NAICS 2-digit Sector
- KSIC 제조업(C)은 중분류 10–34, ISIC/NACE는 10–33, NAICS는 31–33. 매핑 테이블 `industry_category` + `industry_code_mapping` 으로 관리.

16.2 참고 문서

- `.doc/Perplexity-파인튜닝 진행.md` — 8 Stage 원본
- `.doc/Perplexity-ISIC,KSIC,NACE,NAICS 분류 조사 및 정리.md` — 분류 체계 원본
- UN ISIC Rev.4 / KOSTAT KSIC(10·11차) / Eurostat NACE Rev.2 / US Census NAICS

16.3 용어집

- **PT / IT**: Pretrained / Instruction-tuned
- **LoRA / QLoRA**: 저랭크 어댑터 / 4bit 양자화 LoRA
- **GGUF**: llama.cpp 진영의 inference 포맷
- **Reaper**: 끊긴 잡과 미접속 사용자 자원을 회수하는 백그라운드 작업
- **HW Caps**: 시스템 하드웨어 능력 스냅샷